

PARALLEL FILE MANAGEMENT SYSTEM

**Ayesha Zahid
Ayesha Naeem
Ahmad Bilal
Zaraiz**

12/05/2024

—

Parallel & Distributed Computing

—

Sir Haseeb Arshad

Introduction

The Parallel Patient Record Management System project aims to efficiently manage patient records across multiple nodes in a parallel computing environment. The system handles fundamental operations such as record distribution, read, update, delete, backup, recovery, search, and more. The development process focuses on modularity, essential features, and reliability in distributed computing, leveraging MPI for communication and coordination among nodes.

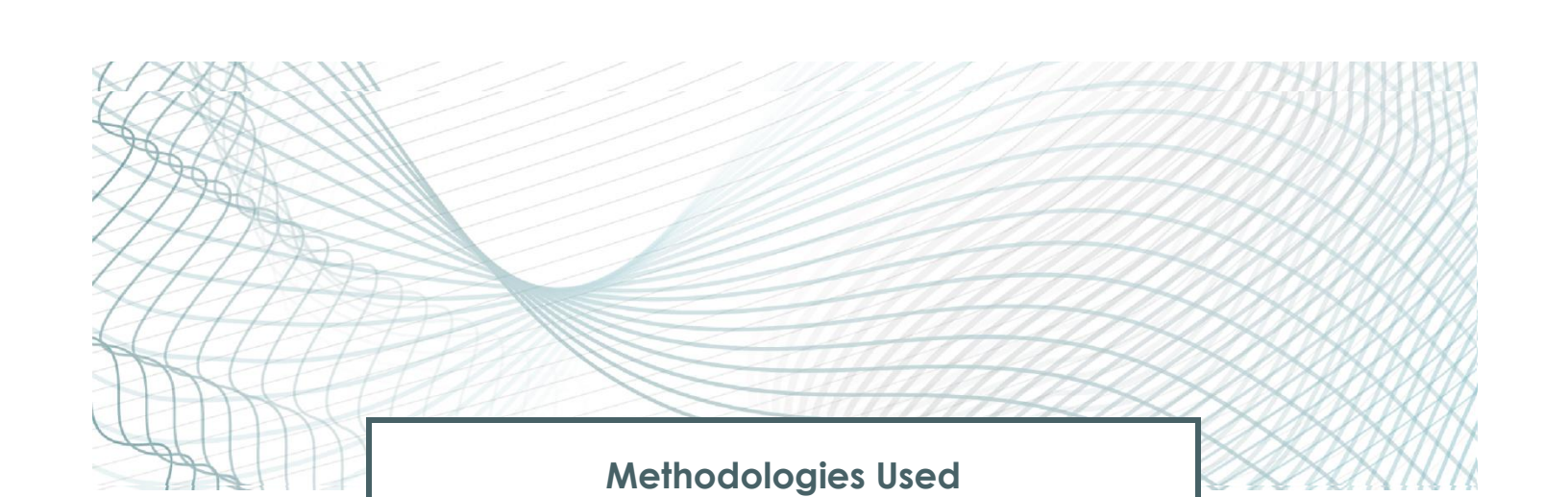




Development Process

The project followed a modular approach, focusing on core functionality in the following stages:

1. **Initialization and Configuration:** The system is prepared for managing patient records and communication among nodes by initializing the MPI environment, parsing patient configuration files, and establishing MPI communicators.
2. **Patient Distribution:** Patient records were divided into manageable chunks and distributed evenly among nodes using MPI for communication. Mechanisms were implemented to handle patient record chunks efficiently and monitor system performance.
3. **Implementing Operations:** Functions for adding, updating, and deleting patient records were created. Read operations were enabled to access patient records distributed across multiple nodes, ensuring data consistency and reliability.
4. **Backup and Recovery:** Scheduled backups of patient records were implemented to prevent data loss, and procedures for restoring patient records from backups were developed.
5. **Distributed Search:** An indexing system was developed, enabling distributed search across patient records stored on multiple nodes using MPI for coordination.
6. **Logging:** System operations and errors were logged for auditing and debugging purposes, and basic system monitoring functionalities were implemented.
7. **Load Balancing:** Algorithms were developed to distribute patient record processing evenly across nodes, utilizing MPI for gathering performance metrics and triggering load rebalancing when necessary.
8. **Parallel Compression and Decompression:** Parallel algorithms for compressing and decompressing patient records were designed, reducing network load and storage requirements.



Methodologies Used

1. **Parallel Computing**

MPI (Message Passing Interface) was used for parallel and distributed computing, enabling efficient communication and coordination among nodes.

2. **Software Engineering Principles**

Modularity and maintainability were emphasized throughout the project, ensuring a simple interface for users to interact with the system and enhancing usability and accessibility.

3. **Problem-solving and Debugging**

Various problem-solving and debugging techniques were applied to ensure the reliability and efficiency of the distributed system.

4. **Project Management and Collaboration**

Collaboration and communication among team members were essential for managing tasks, meeting deadlines, and ensuring a cohesive final product.

5. **Continuous Learning**

An adaptable and continuous learning mindset was adopted to stay updated on the latest technologies and best practices in parallel computing and distributed systems.

Conclusion

The Parallel File Management System project successfully demonstrated proficiency in MPI programming, software engineering principles, and problem-solving abilities. The project's development process and methodologies used provided valuable experience in project management, collaboration, and adaptability for future projects. The system's modular design and efficient parallel computing techniques ensure reliable operation in distributed computing, handling fundamental patient record operations and providing flexibility for future enhancements.